Лабораторная работа 3

Attention и моделирование длинных последовательностей

Positional representations, extrapolation, complexity и local attention

Курс «Нейронные сети»

Редакция 4 от 8 октября 2026 г.

1 Введение

Внимание (attention) сопоставляет каждую позицию последовательности со всеми остальными. При длине 64 матрица одной головы содержит 4 096 чисел. При длине 256 чисел уже 65 536, то есть в шестнадцать раз больше. Модель при этом часто учили только на коротких входах. Качество, память и перенос навыка могут сломаться по разным причинам.

Самовнимание (self-attention) без сведений о позиции видит набор токенов, а не их порядок. Фразы «кот сидел на ковре» и «ковре на сидел кот» для него один набор. Код позиции — часть модели, а не мелочь реализации.

Запуски ведутся в проекте из ЛР0. У каждого есть `EXP_ID`, `RUN_ID`, паспорт и итоговая конфигурация. Числа в отчёте сопровождаются ссылками на `RUN_ID`.

2 Цель работы

Цель — отделить три причины деградации модели на длине больше обучающей. Итог — кривая качества и измеренная стоимость, а не одна цифра на привычной длине.

Что ограничивает модель с вниманием на длине больше обучающей: позиция, память или сама задача?

1. Код позиции не задан или не выучен за пределами обучающей длины.
2. Время и память внимания растут быстрее, чем сама длина.
3. Зависимость в задаче не переносится на новые позиции.

3 Что нужно знать заранее

Если какой-то пункт незнаком, восполните пробел по указанному источнику.

- Остаточный блок и нормализация — ЛР1.

- Внимание, кодировщик и декодировщик — [1, разд. 8.1], [9, 7].
- Долгие зависимости в последовательностях — [2, разд. 10.7].
- Текст как последовательность признаков — [2, разд. 12.4].
- Слойная нормализация (LayerNorm) — [10, 5].
- Реализация внимания в PyTorch — [3, 4].
- Позиционные коды — заметка Hugging Face [8].
- Проект для экспериментов из ЛР0.

4 Результаты обучения

После выполнения работы вы сможете:

1. объяснять, почему без позиции модель не различает порядок токенов;
2. сравнивать обучаемые абсолютные, синусоидальные и поворотные коды;
3. строить кривую качества за пределами обучающей длины;
4. измерять рост времени и памяти и связывать его с реализацией;
5. считать энтропию и массу внимания, не принимая карту за объяснение;
6. делать вывод по парным разностям трёх общих запусков.

5 Модели и обозначения

5.1 Самовнимание и блок

Модель — небольшой трансформер (Transformer). Собирать его с нуля во всех деталях не требуется. Нужны свой код позиции, доступ к матрице весов и фиксированный блок.

Вход слоя — матрица X размера $L \times d$. Строка x_m — вектор токена на позиции $m = 0, \ldots, L-1$. Голов h, размер головы $d_k = d/h$. Для одной головы масштабированное скалярное внимание задаётся так [9]:

$$\begin{aligned} Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \\ A = \operatorname{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + C\right), \quad Y = AV. \end{aligned} \tag{3.1}$$

Здесь W_Q, W_K и W_V — проекции, softmax берётся по строке. $C_{ij} = -\infty$ запрещает позиции i смотреть на j, иначе $C_{ij} = 0$. Маска прибавляется *до* softmax: запрещённый вес становится нулём и не входит в нормировку. A_{ij} — доля, с которой позиция i смотрит на j. Y — выход головы.

Множитель $1/\sqrt{d_k}$ удерживает скалярное произведение от роста с шириной головы. Если компоненты q и k независимы, с нулевым средним и единичной дисперсией, то

$$\operatorname{Var}(q^\top k) = d_k. \tag{3.2}$$

Без делителя softmax при большом d_k почти всю массу отдаёт одному ключу.

Несколько голов склеиваются и умножаются на выходную проекцию. Блок с нормализацией до ветви (pre-norm) имеет вид [9, 11]

$$X' = X + \mathrm{MHSA}(\mathrm{LN}(X)), \qquad X^+ = X' + \mathrm{MLP}(\mathrm{LN}(X')). \tag{3.3}$$

LN — слойная нормализация по признакам токена [10]. MLP — два линейных слоя, отдельные для каждой позиции. Остаток прибавляет вход блока к результату ветви.

Подробнее: [1, разд. 8.1]; [9, 7, 3, 4, 10, 11].

5.2 Позиция и перестановки

Пусть Π — матрица перестановки и маски нет ($C = 0$). Без кода позиции

$$\mathrm{Attn}(\Pi X) = \Pi\ \mathrm{Attn}(X). \tag{3.4}$$

Проекции переставляют строки вместе с токенами, а softmax по строке с этим согласован. Та же перестановка проходит через LN и MLP. Модель видит набор токенов. Каузальная маска сама нарушает (3.4) и тем самым несёт порядок [13]. В основных опытах маски нет, пакет одной длины, дополнения нет.

Проверка на копировании ещё проще. Все пустые позиции имеют один и тот же токен. Без позиции у них на каждом слое один и тот же вектор. Разные символы фрагмента они выдать не могут.

На поиске по содержанию склейте две раскладки. В первой пары $4 \to 9$ и $1 \to 7$, запрос 1, ответ 7. Во второй те же токены, но пары $4 \to 7$ и $1 \to 9$, ответ 9. Набор токенов совпал, значит выход без позиции совпал. Хотя бы один из двух ответов неверен. Если ключ и значение склеены в один токен, набор уже хранит пару. Тогда позиция не нужна, и проверка пустая.

Learned absolute embeddings. К токену прибавляется строка таблицы (learned absolute embeddings):

$$z_m = x_m + p_m, \qquad m = 0, \ldots, L_{\max} - 1. \tag{3.5}$$

Таблица P имеет $L_{\max}$ строк. В этой работе $L_{\max} = L_0$: ровно те позиции, что встречаются в обучении. Строки за пределом L_0 не создавайте. Иначе они останутся случайными и не обновятся.

Для длины $L > L_{\max}$ выберите одно правило и запишите его в конфигурацию.

1. Правило P_1, расширение. Новые строки случайны и не обучаются. Так виден отказ на необученных позициях.
2. Правило P_2, интерполяция. Новый индекс сжимается в обученный диапазон. При $L > 1$
$$t = m\,\frac{L_{\max} - 1}{L - 1}, \qquad \alpha = t - \lfloor t \rfloor, \qquad \tilde{p}_m = (1 - \alpha)\, p_{\lfloor t \rfloor} + \alpha\, p_{\lceil t \rceil}. \tag{3.6}$$
При $L = L_{\max}$ получается $t = m$, то есть исходная строка. Соседние токены длинной последовательности получают более близкие коды, чем при обучении. Сжатие индекса в обученный диапазон и пересчёт сетки вложений — тот же приём [14, 15].
3. Правило P_3, отказ. За пределом таблицы метод не определён, точки на графике нет.

На кривых основного опыта для обучаемой таблицы стоит правило P_2. Правило P_1 нужно одной точкой при $L = 2L_0$. Правило указано в подписи графика и в итоговой конфигурации.

Sinusoidal encoding. Позиция m задаётся синусами и косинусами (sinusoidal encoding) [9]:

$$\mathrm{PE}(m, 2i) = \sin\!\big(m/10000^{2i/d}\big), \qquad \mathrm{PE}(m, 2i+1) = \cos\!\big(m/10000^{2i/d}\big). \tag{3.7}$$

Индекс компоненты $i = 0, \ldots, d/2 - 1$. Вектор определён при любом m, в том числе при $m \geq L_0$. Сдвиг на k позиций вращает каждую пару синус–косинус на угол, который зависит только от k. Это не значит, что модель пользуется сдвигом за пределами обучения.

Rotary Position Embedding (RoPE). Поворотное кодирование (RoPE) вращает уже готовые запрос и ключ, а не вектор токена [12]. На плоскости с номером $i = 1, \ldots, d_k/2$ матрица поворота такова:

$$R_m^{(i)} = \begin{pmatrix} \cos m\theta_i & -\sin m\theta_i \\ \sin m\theta_i & \cos m\theta_i \end{pmatrix}, \qquad \theta_i = 10000^{-2(i-1)/d_k}. \tag{3.8}$$

Отсюда скалярное произведение зависит от позиций только через разность:

$$(R_m q_m)^\top (R_n k_n) = q_m^\top R_{n-m} k_n. \tag{3.9}$$

Для первой плоскости $\theta_1 = 1$, то есть угол равен номеру позиции в радианах. На позиции 256 он в четыре раза больше, чем на позиции 64. Такой угол при обучении до 64 не встречался. Относительный вид (3.9) поэтому не даёт экстраполяции по длине сам по себе [13]. Экстраполяция по длине (length extrapolation) — это сохранение качества при $L > L_0$.

Абсолютные коды, включая синусоиду и отказ от позиции, подключаются так:

```
x = position_module(x, positions)
```

Поворотный код подключается иначе:

```
q, k = position_module(q, k, positions)
```

Подробнее: [8, 9, 12, 13, 14, 15].

5.3 Стоимость, диагностики и кривые

Один слой внимания стоит

$$T(L) = O(L^2 d + L d^2), \qquad \mathrm{Mem}_A(L) = O(hL^2). \tag{3.10}$$

Первое слагаемое времени — произведения $QK^\top$ и AV. Второе — проекции. Память $O(hL^2)$ относится только к явному хранению матрицы A. Если матрица не хранится, дополнительная память внимания линейна по L при том же порядке времени [16]. Показатель по памяти поэтому зависит от реализации. Его нельзя переносить с одной реализации на другую.

Для строки i матрицы A считаются энтропия и среднее расстояние:

$$H_i = -\sum_{j=1}^{L} A_{ij} \ln(A_{ij} + \varepsilon), \qquad D_i = \sum_{j=1}^{L} A_{ij} \, |i - j|. \tag{3.11}$$

Логарифм натуральный. У равномерной строки $H_i = \ln L$. У строки, вся масса которой на одном токене, энтропия близка к нулю. Доля массы в окне радиуса w и масса на целевом токене:

$$\mu_i(w) = \sum_{|i-j| \leq w} A_{ij}, \qquad \mu_i^{\mathrm{tgt}} = A_{i,j^*}. \tag{3.12}$$

j^* — позиция верного значения. Эти числа — диагностика, а не объяснение решения [22]. Яркая связь на карте может быть не нужна модели.

$M(L)$ — доля верных целевых токенов на длине L. Отношение к обучающей длине и разность методов:

$$ER(L) = \frac{M(L)}{M(L_0) + \varepsilon}, \qquad \Delta_{\text{pos}}(L) = M_{\text{method}}(L) - M_{\text{abs}}(L). \tag{3.13}$$

M_{abs} — обучаемые абсолютные вложения с правилом P_2. $\varepsilon = 10^{-8}$. Наклон времени или памяти в логарифмических координатах:

$$\alpha = \frac{\log T(L_2) - \log T(L_1)}{\log L_2 - \log L_1}. \tag{3.14}$$

Основание логарифма любое: в отношении оно сокращается. Для всей сетки α — наклон прямой регрессии. У чисто квадратичной зависимости удвоение длины умножает величину на 4, и $\alpha = 2$. У линейной удвоение умножает её на 2, и $\alpha = 1$. Допуск 20% вокруг двойки даёт интервал от 1,6 до 2,4. Это рост при удвоении длины примерно от $2^{1,6} \approx 3{,}03$ до $2^{2,4} \approx 5{,}28$. Вокруг единицы тот же допуск даёт интервал от 0,8 до 1,2.

Подробнее: [16, 22].

5.4 Сводка обозначений

Обозначение	Смысл
L, L_0, $L_{\max}$, m	длина входа, максимум обучения, размер таблицы, индекс
d, h, d_k, N	ширина, число голов, размер головы, число блоков
X, Q, K, V, A, C	токены, запрос, ключ, значение, веса, маска
p_m, PE, R_m, θ_i	абсолютный код, синусоида, поворот и его угол
H_i, D_i, μ_i	энтропия, среднее расстояние, масса в окне или на цели
$M(L)$, $ER(L)$, Δ_{pos}	точность, отношение к L_0, разность методов
α, Δ_s, s_Δ	наклон, парная разность и её разброс
w, R, ε	радиус окна, досягаемость, малая константа

6 Данные и оборудование

Данные синтетические: их порождает генератор, а не внешний корпус [18]. Параметры генератора и есть манифест набора. Роли — `train`, `val`, `cal` и `test`, у каждой свой `seed`. Обучение читает только `train`. Скорость обучения выбирается по `val`. Итоговые $M(L)$ считаются по `test`. Роль `cal` порождается и не читается до ЛР8. Смена тестовой длины строит новые последовательности и не меняет `train`. На каждую тестовую длину нужно не меньше 200 последовательностей. Словарь содержимого — 64 символа с номерами от 0 до 63. Случайный ответ имеет долю $1/64 \approx 0{,}016$.

Длина L везде — число токенов во входе. Матрица внимания имеет размер $L \times L$. В обучении L равномерно берётся из целых от $L_0/2$ до L_0. Длиннее L_0 обучение не заглядывает.

Задача	Пример входа	Ответ
Копирование	4, 1, 7, пусто, пусто, пусто	4, 1, 7
Поиск	ключ, 4, значение, 9, ключ, 1, значение, 7, запрос, 1	7
Иголка	8, 1, метка, 5, 3, запрос	5

- Копирование [19]. Если L нечётно, уменьшите её на один. Первая половина — случайный фрагмент, вторая — пустые позиции. На пустых местах нужно повторить фрагмент по порядку. Потеря считается только там.
- Поиск по содержанию [20]. Число пар $n = \min(32, \max(2, \lfloor L/8 \rfloor))$. Пара — это метка ключа, ключ, метка значения, значение. Ключи в одной последовательности не повторяются. Запрос — два последних токена: метка запроса и один из ключей. Ответ — значение этого ключа, читается с последней позиции. Остальные места занимает фон из случайных символов. Целевая пара стоит в случайном месте.
- Иголка [21]. Фон — случайные символы, не один повтор. Метка одна, ответ — символ сразу после неё. Место метки равномерно среди позиций, где после неё ещё есть символ. Последний токен — метка запроса, с него читается ответ.

Потеря — перекрёстная энтропия только на целевых позициях. Остальные позиции в потерю не входят. Чтение ответа с одной и той же позиции не добавляет порядка само по себе.

Оборудование. Процессора достаточно при ширине 128 и глубине не больше 4. Тестовые длины — до 1024 токенов. Видеокарта ускоряет те же запуски, протокол не меняется. Если длина $8L_0$ не помещается в память, остановите сетку раньше и запишите причину.

7 Индивидуальный профиль

Профиль ЛР3 считается функцией `lab_pick` из ЛР1 с номером работы 3. Добавьте три поля:

```
def lab_pick(student_id: str, lab: int, a: int, b: int, options: list):
    hk = hashlib.sha256(f"{student_id}:lab{lab}".encode("utf-8")).hexdigest()
    return options[int(hk[a:b], 16) % len(options)]

profile["lab3_task"] = lab_pick(sid, 3, 0, 2, ["copy", "retrieval", "needle"])
profile["lab3_train_max"] = lab_pick(sid, 3, 2, 4, [64, 128, 256])
profile["lab3_depth"] = lab_pick(sid, 3, 4, 6, [2, 4])
```

Здесь `sid` — номер зачётки, `profile` — словарь профиля из ЛР0. `copy` — копирование, `retrieval` — поиск, `needle` — иголка. `lab3_train_max` — это L_0. `lab3_depth` — число блоков N. Ширина $d = 128$, голов $h = 4$, словарь и сетка длин общие. Смена профиля не требует правки цикла обучения.

8 Задания

8.1 Задание 1. Реестр позиционных представлений

Что сделать.

1. Реализуйте четыре режима: без позиции, обучаемая таблица, синусоида (3.7), поворот (3.8). Режим выбирается только конфигурацией.
2. Для таблицы реализуйте правила P_1, P_2 и P_3. При $L = L_{\max}$ правило P_2 возвращает исходные строки, без пересчёта в вещественных числах.
3. Абсолютные режимы прибавляют код к вектору токена (3.5). Поворотный режим вращает запрос и ключ каждой головы.
4. Не добавляйте порядок побочно. В проверке без позиции нет каузальной маски, нет дополнения и длины в пакете равны.

Что должно получиться. Смена режима не требует правки цикла обучения.

Как проверить себя. Перестановка входа переставляет выход только без позиции и без маски (тест 2). Синусоида совпадает с формулой за пределом L_0 (тест 3). Поворот сохраняет размеры и зависит от разности позиций (тест 4). Правило P_2 при $L = L_{\max}$ не меняет таблицу (тест 5).

8.2 Задание 2. Профилировщик последовательностей

Что сделать.

1. Для заданной длины измерьте время прямого прохода, время одного шага обучения и пик памяти. Пропускная способность — число токенов в секунду, посчитайте её из времени прямого прохода.
2. Запишите имя реализации внимания и число токенов L. Для диагностик нужна явная матрица A: её даёт математическая реализация или модуль многоголового внимания с запросом весов [3, 4].
3. Сделайте не меньше 10 прогревочных вызовов и не включайте их в среднее. На видеокарте синхронизируйте выполнение перед замером [6]. Перед замером пика обнулите счётчик. На видеокарте берите пик распределителя. На процессоре берите пик `tracemalloc` за измеряемый вызов.
4. Не подставляйте вместо замера выражение $4hL^2$. Размер матрицы на бумаге всегда даёт $\alpha = 2$ и не проверяет реализацию.

Что должно получиться. Один вызов возвращает времена, пик памяти, имя реализации и L.

Как проверить себя. Среднее не меняется, если увеличить только число прогревочных вызовов. Имя реализации в паспорте совпадает с тем, что реально вызвано.

8.3 Задание 3. Диагностика внимания

Что сделать.

1. Для выбранных блоков и каждой головы считайте H_i, D_i, $\mu_i(16)$ и максимальный вес строки (3.11), (3.12).
2. На поиске и иголке дополнительно считайте μ^{tgt} в позиции ответа. j^* — позиция верного значения.

3. Усредняйте по объектам пакета. Головы не смешивайте. Отдельно сохраните энтропию в позиции ответа.

4. Запишите сводку в `summary.json`, например:

```
"attention": {"entropy": 1.2, "distance": 14.0,
              "target_mass": 0.4, "backend": "math"}
```

Что должно получиться. По сохранённому запуску видно, куда смотрела каждая голова. Карта при этом не объявляется причиной ответа.

Как проверить себя. Сумма разрешённых весов равна единице, запрещённые веса нулевые (тест 1). У равномерной строки энтропия совпадает с $\ln L$ с точностью 10^{-5}.

Что добавляется в общий проект. В `src/` добавляются `models/sequence/positional.py`, `models/sequence/transformer.py`, `profiling/sequence_profiler.py`, `diagnostics/attention_stats.py`, `diagnostics/extrapolation.py` и `data/synthetic_sequence.py`. Ими пользуются опыты этой работы. Генератор при фиксированном `seed` повторяем (тест 6).

9 Эксперименты

Перед каждым опытом запишите гипотезу: что ждёте и почему, со ссылкой на формулу раздела 5. E1 и E2 — поисковые, достаточно одного `seed`. Главный вывод проверяется в E3 на трёх общих `seed`. Если результат разошёлся с гипотезой, это допустимо. Укажите, какое предположение не выполнилось.

Общие условия. Ширина $d = 128$, голов 4, внутренний слой блока — $2d$, нормализация до ветви, глубина из профиля. Каузальной маски нет. Оптимизатор AdamW. Скорость обучения выберите один раз по `val` на синусоидальном коде. Сетка — $3 \cdot 10^{-4}$, 10^{-3} и $3 \cdot 10^{-3}$. Длины больше L_0 в этом выборе не участвуют. Дальше скорость не меняйте. Пакет обучения — 32 последовательности. Начните с $2 \cdot 10^3$ шагов. На `val` при длине L_0 сравните лучший код с режимом без позиции. Если отрыв меньше 0,20, удвойте число шагов. Остановитесь, когда разрыв достигнут или когда сделано $8 \cdot 10^3$ шагов. Число шагов одинаково у всех режимов позиции. Один и тот же `seed` задаёт и начальные веса, и последовательности.

9.1 Эксперимент E1. Позиция и экстраполяция по длине

Вопрос. Какой код позиции сохраняет качество после выхода за обучающую длину, и сохраняет ли его режим без позиции?

Что сделать. Обучите четыре режима: без позиции, таблица с правилом P_2, синусоида, поворот. Задача и L_0 — из профиля. На `test` возьмите длины L_0, $2L_0$, $4L_0$ и $8L_0$, пока хватает памяти. Доучивать на этих длинах нельзя. Для таблицы добавьте одну оценку при $L = 2L_0$ с правилом P_1. Постройте график $M(L)$ для четырёх режимов. В подписи укажите правило таблицы. Постройте $ER(L)$ по (3.13) для трёх кодов позиции. Для режима без позиции ER не стройте, если $M(L_0)$ не выше случайного уровня. На том же графике или отдельно покажите точку P_1.

Порог приёмки. На `val` при длине L_0 точность без позиции не выше $1/64 + 0{,}05$. Хотя бы один код позиции выше него не меньше чем на 0,20. Если после $8 \cdot 10^3$ шагов разрыва нет, опыт принимается только с разбором: утечка порядка, ошибка генератора или задача не выучивается. Кривые $M(L)$ и точка P_1 всё равно нужны. Правило и реализация внимания записаны в итоговой конфигурации.

Граница применимости. Для каждого кода укажите наименьшую длину больше L_0, где точность падает до $1/64 + 0{,}05$. Либо укажите первую длину, которая не поместилась в память. Напишите, что из этого случилось. Если падения нет и память не кончилась, граница лежит дальше сетки. Выход за L_0 — место, где код позиции ещё определён формулой, но уже не обязан сохранять навык. Для синусоиды и поворота это прямо следует из (3.7) и (3.9).

9.2 Эксперимент E2. Квадратичный рост стоимости

Вопрос. Растут ли время и память как квадрат длины именно в вашей реализации внимания?

Что сделать. Обучение не нужно: веса случайны. Пакет профилирования — 4 последовательности. Возьмите не меньше пяти длин из сетки 64, 128, 256, 512, 1024. Если длинная точка не запускается, замените её промежуточной и запишите отказ по памяти. Измерьте время прямого прохода, время шага обучения и пик памяти. Постройте два графика в логарифмических координатах: время от L и память от L. Нанесите прямую регрессии и подпишите α по (3.14). Отдельно посчитайте α по двум наибольшим длинам. Если доступны две реализации, постройте две кривые памяти, а не одну общую. Имя реализации — в подписи.

Порог приёмки. Сравнение наклона с двойкой корректно, когда величина при удвоении длины вырастает не меньше чем в 3,03 раза. Тогда для времени прямого прохода и для памяти нужно $1{,}6 \le \alpha \le 2{,}4$. Для реализации без явной матрицы A память сравнивайте с единицей: $0{,}8 \le \alpha \le 1{,}2$. С двойкой её не сравнивайте. Если ни один из этих режимов не выполнен, разложите замер по двум слагаемым (3.10) и покажите, какое сдвинуло наклон. Такой отчёт принимается. Подстановка $4hL^2$ вместо пика памяти не принимается.

Граница применимости. При малых L слагаемое проекций Ld^2 сравнимо с $L^2 d$, и наклон меньше двух. Это граница режима измерения, а не опровержение формулы. На процессоре пик может почти не расти из-за кэша распределителя. Тогда приведите размер крупнейшего тензора отдельно и не объявляйте память линейной.

9.3 Эксперимент E3. Основное сравнение

Вопрос. Выдерживает ли главный вывод смену `seed`?

Что сделать. По E1 выберите две конфигурации. Между ними меняется ровно один фактор: способ кодировать позицию. Остальное зафиксировано, включая правило P_2 для таблицы. Пример утверждения: поворот сохраняет $M(4L_0)$ лучше таблицы. До запусков запишите в `hypothesis.md` и зафиксируйте в Git:

1. чем различаются конфигурации;
2. что зафиксировано, включая правило таблицы и реализацию внимания;
3. ожидаемый знак разности и длину $L^* > L_0$, на которой она считается;
4. какой результат опровергнет гипотезу.

Обучите обе конфигурации с тремя общими `seed` $s = 1, 2, 3$. Если конфигурация уже есть в E1, её запуск с первым `seed` можно переиспользовать. Метрика — $M(L^*)$ на `test`. Парные разности:

$$\Delta_s = M(\mathrm{A}, s) - M(\mathrm{B}, s), \qquad \bar{\Delta} = \frac{1}{3}\sum_{s=1}^{3} \Delta_s,$$
$$s_\Delta = \sqrt{\frac{1}{2}\sum_{s=1}^{3}(\Delta_s - \bar{\Delta})^2}. \tag{3.15}$$

Здесь $M(\mathrm{A}, s)$ — метрика первой конфигурации при `seed` s. $\bar{\Delta}$ — средняя разность, s_Δ — её стандартное отклонение.

Порог приёмки. Вывод «конфигурации различаются» установлен, если все три Δ_s одного знака и $|\bar{\Delta}| \geq 2s_\Delta$. Иначе вывод — «различие не установлено». В отчёте все три Δ_s, а не только среднее. Расхождение с гипотезой не снижает оценку, если оно объяснено.

Граница применимости. Три `seed` описывают разброс на этом генераторе и этой длине. Перенос на естественный текст они не дают.

10 Контрфактические вопросы

Ответ опирается на ваши числа и ссылки на `RUN_ID`. Если нужен ещё один запуск без смены протокола, сделайте его.

1. По кривой E1 возьмите $ER(L)$ синусоиды на самой большой измеренной длине. Почему формула (3.7), определённая при любом m, не гарантирует тот же $M(L)$, что на L_0?
2. Во сколько раз должно вырасти число элементов матрицы A, если L увеличить в четыре раза? Сравните с измеренным отношением пика памяти и с вашим α.
3. Приведите свои $M(2L_0)$ для правил P_1 и P_2. Какое различие режимов видно по этим двум числам?
4. Если на L^* поворот лучше таблицы, значит ли это, что навык не зависит от длины? Ответьте по всей кривой $M(L)$ и по знакам Δ_s.
5. Удвойте ширину d при фиксированном L и повторите замер времени прямого прохода. Слагаемое $L^2 d$ должно вырасти вдвое, слагаемое Ld^2 — вчетверо. К какому из множителей ближе ваш замер и почему?

11 Обязательные тесты

1. Сумма разрешённых весов строки равна 1 с точностью 10^{-5}. Запрещённые позиции имеют вес 0.
2. Без позиции и без маски перестановка входа переставляет выход. Максимальная абсолютная ошибка меньше 10^{-5}.
3. Синусоида для позиции $m \geq L_0$ совпадает с (3.7) и имеет длину d.
4. Поворот сохраняет размеры Q и K. При одной и той же разности позиций скалярные произведения совпадают с точностью 10^{-5}.
5. Правило P_2 при $L = L_{\max}$ возвращает исходную таблицу с точностью 10^{-6}.
6. Генератор при фиксированном `seed` повторяем. В поиске ключ запроса встречается один раз. В иголке метка одна. Смена тестовой длины не меняет сохранённые обучающие последовательности.

12 Что сдать

Сдаются модули заданий 1–3, конфигурации E1–E3, файл `hypothesis.md`, тесты и отчёт. Отчёт состоит из шести частей:

1. **Профиль и протокол:** задача, L_0, глубина, скорость обучения, число шагов, правило таблицы, реализация внимания.
2. **E1:** графики $M(L)$ и $ER(L)$, точка P_1, порог режима без позиции, граница по каждому коду.
3. **E2:** графики в логарифмических координатах, значения α, имя реализации.
4. **Диагностики:** энтропия, расстояние и масса на цели для одной обученной модели на L_0 и на $2L_0$, по головам.
5. **E3:** гипотеза, все Δ_s, вывод.
6. **Ответы на контрфактические вопросы.**

Каждое число сопровождается ссылкой на `RUN_ID`. Графики строятся из сохранённых результатов.

13 Критерии завершения и оценивания

Работа завершена, если одновременно:

- опыты E1–E3 выполнены, у каждого результата есть `RUN_ID`;
- тесты проходят;
- файл `hypothesis.md` зафиксирован в Git до запусков E3;
- правило таблицы и реализация внимания записаны в итоговой конфигурации;
- диагностики посчитаны и прокомментированы;
- на все контрфактические вопросы даны ответы с собственными числами.

Веса частей оценки те же, что в ЛР0: реализация 0,15, экспериментальный протокол 0,30, диагностика 0,30, перенос 0,25.

14 Дополнительные задания

1. **Локальное внимание.** Ограничьте взгляд окном радиуса w [17]:

$$C_{ij} = \begin{cases} 0, & |i-j| \le w, \\ -\infty, & |i-j| > w. \end{cases} \tag{3.16}$$

   После N блоков сведения о токене проходят не дальше чем на

$$R \le Nw \tag{3.17}$$

   позиций. Это верхняя оценка досягаемости, а не обещание сохранить сведения на этом пути. Возьмите $w \in \{4, 16, 64\}$ и полное внимание. Для одного окна обучите модель с теми же условиями, что в E1. Для остальных окон маску можно надеть на уже обученную модель без дообучения. Постройте таблицу: точность, время, память. Если маска надета на плотную матрицу, память может не упасть. Напишите, что вышло у вас. Модуль положите в `models/sequence/local_attention.py`.

2. **Расстояние зависимости.** На задаче профиля меняйте расстояние между запросом и целью. Возьмите не меньше четырёх расстояний и постройте $M(d)$. Если сделано локальное окно, отметьте на графике границу Nw. Сравните, где качество падает, с этой границей. Если падения нет, напишите, что граница на сетке не достигнута.

3. **Возмущение карты внимания.** На одной обученной модели сделайте два вмешательства без дообучения [23, 24]. Первое — обнулите веса дальше расстояния 16 и заново нормируйте строку. Второе — замените одну голову на равномерное внимание по разрешённым позициям. Измерьте изменение M. Если качество не упало, яркая связь не была нужна [22].

15 Контрольные вопросы

1. Почему из (3.4) следует, что без позиции порядок токенов неразличим?
2. Зачем в (3.1) делить скалярное произведение на $\sqrt{d_k}$?
3. Почему рост памяти как L^2 относится не к каждой реализации?
4. Чем абсолютная позиция отличается от относительной в (3.9)?
5. Почему синусоида определена при любом m, но $M(L)$ всё равно может упасть?
6. Почему маску C нужно прибавлять до softmax, а не после?
7. Почему большая масса μ^{tgt} не доказывает, что голова необходима?
8. Почему удачный поиск на синтетике не означает понимания длинного текста?

16 Итог

Длина, на которой хватает памяти, и длина, на которой держится качество, не совпадают. Код позиции может быть определён формулой и всё равно не работать за пределом обучения. Реализация внимания меняет память, даже если формула весов одна и та же. Реестр позиций, профилировщик и диагностики остаются в общем проекте.

Литература

[1] Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. Погружение в мир нейронных сетей. — СПб.: Питер, 2018. — 480 с.

[2] Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение / пер. с англ. А. А. Слинкина. — 2-е изд., испр. — М.: ДМК Пресс, 2018. — 652 с.

[3] PyTorch Documentation: torch.nn.functional.scaled_dot_product_attention. https://docs.pytorch.org/docs/stable/generated/torch.nn.functional.scaled_dot_product_attention.html

[4] PyTorch Documentation: torch.nn.MultiheadAttention. https://docs.pytorch.org/docs/stable/generated/torch.nn.MultiheadAttention.html

[5] PyTorch Documentation: torch.nn.LayerNorm. https://docs.pytorch.org/docs/stable/generated/torch.nn.LayerNorm.html

[6] PyTorch Documentation: torch.utils.benchmark. https://docs.pytorch.org/docs/stable/benchmark_utils.html

[7] Hugging Face LLM Course, глава 1.4 «Как работают трансформеры?». https://huggingface.co/learn/llm-course/ru/chapter1/4

[8] C. Fleetwood. You could have designed state of the art positional encoding. Hugging Face Blog, 2024. https://huggingface.co/blog/designing-positional-encoding

[9] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. Attention is all you need. *Advances in NeurIPS*, 2017.

[10] J. L. Ba, J. R. Kiros, G. E. Hinton. Layer normalization. arXiv:1607.06450, 2016.

[11] R. Xiong, Y. Yang, D. He, K. Zheng, S. Zheng, C. Xing, H. Zhang, Y. Lan, L. Wang, T.-Y. Liu. On layer normalization in the transformer architecture. *Proc. ICML*, PMLR 119:10524–10533, 2020.

[12] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, Y. Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

[13] A. Kazemnejad, I. Padhi, K. N. Ramamurthy, P. Das, S. Reddy. The impact of positional encoding on length generalization in Transformers. *Advances in NeurIPS*, 2023.

[14] S. Chen, S. Wong, L. Chen, Y. Tian. Extending context window of large language models via positional interpolation. arXiv:2306.15595, 2023.

[15] A. Dosovitskiy, L. Beyer, A. Kolesnikov et al. An image is worth 16x16 words: Transformers for image recognition at scale. *Proc. ICLR*, 2021.

[16] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. *Advances in NeurIPS*, 2022.

[17] I. Beltagy, M. E. Peters, A. Cohan. Longformer: The long-document transformer. arXiv:2004.05150, 2020.

[18] Y. Tay, M. Dehghani, S. Abnar, Y. Shen, D. Bahri, P. Pham, J. Rao, L. Yang, S. Ruder, D. Metzler. Long Range Arena: A benchmark for efficient transformers. *Proc. ICLR*, 2021.

[19] A. Graves, G. Wayne, I. Danihelka. Neural Turing machines. arXiv:1410.5401, 2014.

[20] J. Ba, G. Hinton, V. Mnih, J. Z. Leibo, C. Ionescu. Using fast weights to attend to the recent past. *Advances in NeurIPS*, 2016.

[21] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, P. Liang. Lost in the middle: How language models use long contexts. *Transactions of the ACL*, 12:157–173, 2024.

[22] S. Jain, B. C. Wallace. Attention is not explanation. *Proc. NAACL-HLT*, pp. 3543–3556, 2019.

[23] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, I. Titov. Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned. *Proc. ACL*, pp. 5797–5808, 2019.

[24] P. Michel, O. Levy, G. Neubig. Are sixteen heads really better than one? *Advances in NeurIPS*, 2019.